SOCIETY MANAGEMENT SYSTEM

Final-Year Project Report

A beginner-friendly web application using HTML, CSS and JavaScript with a rule-based maintenance priority assistant

Project Area	Web Development / AI-inspired Automation
Difficulty	Easy / Beginner-friendly
Frontend	HTML5, CSS3, JavaScript
Storage	Browser localStorage
Prepared for	Academic demonstration

Note: This report describes an academic prototype. The maintenance priority feature uses keyword rules and is not a trained machine-learning model.

1. Abstract

The Society Management System is a simple web application designed to help a residential community organize common administrative activities. It provides a resident directory, maintenance request tracking, society notices, and a sample maintenance payment register. A dashboard summarizes residents, open requests, notices, and the total of recorded demo payments.

The application is developed using HTML for page structure, CSS for layout and responsive design, and JavaScript for application logic. Browser localStorage is used to preserve records on the same browser after refresh. A rule-based priority helper checks issue descriptions for selected keywords and suggests High, Medium, or Low priority. This makes the project suitable for a beginner-level demonstration of explainable automation while clearly distinguishing it from a trained machine-learning model.

2. Problem Statement

Small residential communities may track resident details, maintenance complaints, announcements, and maintenance contributions using separate notebooks or message groups. This can make it difficult to find information and follow the status of a complaint.

The project demonstrates a single interface where these basic activities can be recorded and reviewed. It is a learning prototype and does not replace a secure, production-ready management system.

3. Objectives

• Create a clear and responsive dashboard for a residential society.
• Add and remove resident records with basic validation.
• Record maintenance complaints and display suggested priority.
• Publish and view society notices.
• Record sample payments and calculate a total.
• Demonstrate DOM manipulation, events, functions, arrays, objects, and localStorage in JavaScript.

4. Scope and Main Features

Resident directory: records a resident name, flat number, and 10-digit phone number; prevents duplicate flat numbers in the demo.

Maintenance helpdesk: stores the flat and issue description, suggests priority using keywords, and allows an open request to be marked resolved.

Notices: lets a user publish announcements with a title, message, and date.

Payment register: stores a flat number, amount, and date, then calculates the total of demo entries. It does not process real payments.

Dashboard: displays summary counts and the total demo collection.

Persistence: localStorage saves the current data in the same browser. Data is not synchronized to other devices.

5. Technology Stack

HTML5: forms, tables, headings, sections, and accessible page structure.
CSS3: responsive grid layout, colors, spacing, focus states, and mobile styling.
JavaScript: form validation, priority rules, calculations, event handling, rendering, and storage.
localStorage: simple browser-based persistence for the prototype.

6. System Design and Workflow

User opens index.html → dashboard loads sample or saved data → user adds a resident, maintenance request, notice, or payment → JavaScript validates input and updates the data object → data is saved to localStorage → the dashboard and tables are re-rendered.

Maintenance priority workflow: issue text is converted to lowercase → urgent/safety keywords are checked first → maintenance keywords are checked next → otherwise Low priority is suggested → the user/staff should review the recommendation.

7. AI/ML Component

The prototype includes an explainable, rule-based maintenance priority assistant. Example high-priority terms include fire, smoke, electrical, gas leak, emergency, and water leaking. Medium-priority terms include broken, repair, lift, blocked, and not working. If no listed terms appear, the helper suggests Low priority.

Academic honesty: this is keyword-based automation, not machine learning. It has no training dataset, model fitting, or accuracy evaluation. Priority may be wrong when descriptions are unclear, so a human should review every request.

Future genuine ML extension: collect a consented and anonymized set of past complaints labelled by authorized staff; clean and split the data into training and test sets; train a text classifier such as TF-IDF plus logistic regression in Python; evaluate precision, recall, and a confusion matrix; expose the model through a backend API; and let the JavaScript frontend send descriptions to that API. Do not use sensitive resident information as model input.

8. Data Design

The browser data object has four collections: residents (id, name, flat, phone), requests (id, flat, issue, priority, status), notices (id, title, message, date), and payments (id, flat, amount, date). Numeric IDs are generated from the largest current ID plus one. This structure is suitable for demonstrating basic CRUD-like operations, but a real system should use a server database and stronger validation.

9. Setup and Execution

1. Extract the ZIP file to a folder.
2. Open the project folder.
3. Open index.html in a modern browser such as Chrome, Edge, or Firefox.
4. Add sample records using the forms and refresh the page to verify browser storage.
5. Use fictional data only for classroom demonstrations.

No package manager, database server, API key, or internet connection is required for the core functionality. The optional Google Fonts import may fall back to standard fonts when offline.

10. Testing

Test 1 — Add a resident with valid fields: resident appears in the directory and count increases.
Test 2 — Enter a duplicate flat number: application shows a validation message.
Test 3 — Describe an electrical or fire-related issue: helper suggests High priority.
Test 4 — Submit a general issue: helper assigns a priority based on the keyword rules.
Test 5 — Resolve an open request: status changes and open-request count decreases.
Test 6 — Publish a notice: notice appears in the notice list.
Test 7 — Record a positive payment: payment appears and the dashboard total updates.
Test 8 — Refresh the browser: saved data remains in the same browser.

11. Limitations and Security

The project has no authentication, role-based access, remote database, backups, audit log, multi-user synchronization, or payment gateway. Anyone with access to the browser profile may be able to view or alter stored records. localStorage is not suitable for confidential personal information. Use fictional data for demonstrations and do not deploy this version for real residents.

12. Future Enhancements

• Add a backend such as Node.js/Express and a database such as MongoDB.
• Add secure login and resident/admin roles.
• Add complaint status history, filters, and notifications.
• Integrate a real ML classifier through a protected API and evaluate it with labelled examples.
• Add secure online payment integration only through a compliant payment provider.
• Add backups, audit trails, and privacy controls.

13. Conclusion

The Society Management System demonstrates how a simple web interface can organize common residential tasks. It gives a beginner practical experience with forms, responsive design, JavaScript events, validation, data rendering, and browser storage. Its rule-based priority helper provides a transparent introduction to automated classification. A secure backend and a properly trained and evaluated ML model would be necessary for a production-grade AI-enabled system.

Appendix A — Project files

index.html — page structure and forms
style.css — responsive interface styling
script.js — application logic and rule-based priority helper
README.md — setup, features, and viva summary